



School of Future Tech

Case Study Report

on

Case Study 94: Ride Sharing Carpool Management System

(Java Swing)

by

Satvik

Roll No: 150096725106

Java Programming | B.Tech CSE 2025-29 | Semester III

ITM Skills University

Index

1	Introduction to the Case Study
2	Problem Statement / Case Background
3	Case Study Design
4	Java Concepts and Data Structures
5	Implementation Details
6	Application Screenshots
7	Results and Conclusion
8	References

The report follows the supplied reference report structure while keeping the content limited to Case Study 94 and the implemented project.

1. Introduction to the Case Study

Carpooling allows commuters travelling along similar routes to share a ride and divide the travel cost. This Case Study 94 project models that process as a small Java desktop application. It maintains commuter and route information, stores ride offers and requests, matches compatible routes, tracks completed trips and calculates shared ride costs.

2. Problem Statement / Case Background

A carpooling platform needs a system to match commuters traveling similar routes, manage ride offers and requests, and track shared costs among co-passengers. The system should simulate multiple ride requests being matched concurrently.

Objectives

- Maintain commuter and route information.
- Manage ride offers and requests.
- Match commuters with compatible routes.
- Track and split shared trip costs.
- Simulate concurrent ride matching.
- Provide a carpool management GUI.

Expected Modules

1	Commuter Registration
2	Ride Offer / Request
3	Route Matching
4	Cost Splitting
5	Trip Tracking
6	Ride Search
7	Carpool Reports

3. Case Study Design

The implementation is kept as a small single-project Java Swing application. The GUI provides the seven expected modules, while the model/helper classes handle commuter data, rides, requests, matching and cost splitting. Runtime records are maintained using Java collections rather than an external database.

3.1 Class / File Structure

File / Class	Purpose
Main.java	Starts the application and opens the Swing GUI.
CarpoolGUI.java	Main Swing interface and event handling for the seven modules.
Commuter.java	Stores commuter identity and route information.
Ride.java	Stores ride ID, driver, departure time, seats and cost.
RideRequest.java	Stores a commuter's ride request and route.
Rideable.java	Interface defining ride offer/request behaviour.
RouteMatch.java	Checks whether a ride route matches a request.
CostSplit.java	Calculates shared cost per passenger.
RideTask.java	Runnable task used to simulate concurrent ride matching.

3.2 Application Flow

A commuter is registered with a source and destination. A ride can be offered or a ride request can be created. Route Matching compares the request route with available rides. A successful match uses the compatible ride. Cost Splitting divides the total cost among passengers. Completed rides are stored in trip history, while Ride Search and Carpool Reports provide retrieval and summary views.

4. Java Concepts and Data Structures

Java Concept	Application in the Project
Classes & Objects	Commuter, Ride, RideRequest and helper classes.
Constructors	Initialize commuter, ride and request objects.
Interface	Rideable defines ride behaviour implemented by Ride.
Runnable / Thread	RideTask simulates concurrent ride matching.
ArrayList	Stores commuters, ride offers and ride requests.
LinkedList	Maintains completed trip history.
HashMap	Maps Ride ID to Ride for lookup.
TreeMap	Keeps rides ordered using departure-time keys.
CRUD	Ride management includes adding, updating and deleting rides.
Searching	Ride Search checks route information.
Sorting	Rides are sorted by cost and ordered by departure time.
Swing	CarpoolGUI provides the desktop interface.
Exception Handling	Handles invalid input without abrupt termination.
Validation	Checks route details, seats and cost values.

4.1 Main Data Structures

Collection	Purpose
ArrayList	Runtime lists for commuters, rides and requests.
LinkedList	Completed trip history.
HashMap<Integer, Ride>	Ride ID to ride lookup.
TreeMap<String, Ride>	Departure-time based ordering of rides.

5. Implementation Details

5.1 Module Functions

Module	Implemented Function
Commuter Registration	Registers a commuter with name, source and destination.
Ride Offer / Request	Creates ride offers and requests and supports ride management.
Route Matching	Matches a request with a compatible ride route.
Cost Splitting	Divides total ride cost by the number of passengers.
Trip Tracking	Completes rides and stores them in trip history.
Ride Search	Searches active rides by route.
Carpool Reports	Shows commuter, ride, request and completed-trip counts and ride ordering.

5.2 Validation and Exception Handling

Required commuter details are checked and ride seats and cost values are validated. Numeric input is handled with exception handling so invalid input does not terminate the application abruptly.

5.3 Cost Splitting

The total ride cost is divided by the number of passengers. In the tested example, a ₹600 ride shared by three passengers gives ₹200 per person.

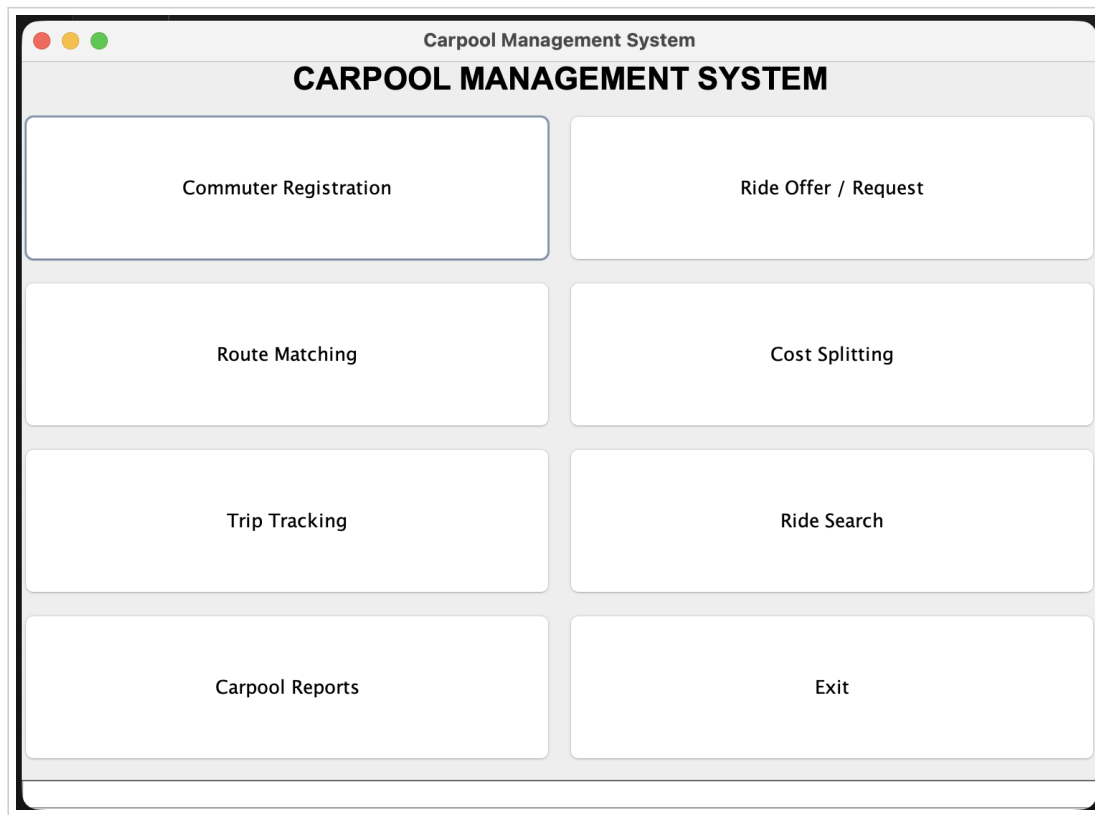
5.4 Concurrent Matching

RideTask implements Runnable and is used with a Thread during route matching to simulate concurrent ride matching as required by the case study.

6. Application Screenshots

The screenshots below show the tested Swing application workflow using the project data.

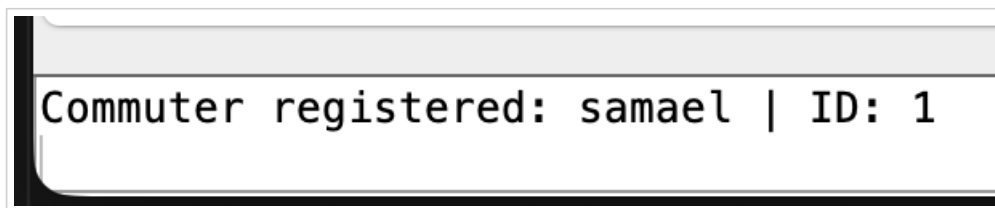
Figure 1: Main Carpool Management GUI



Main Swing window containing the seven modules specified in Case Study 94.

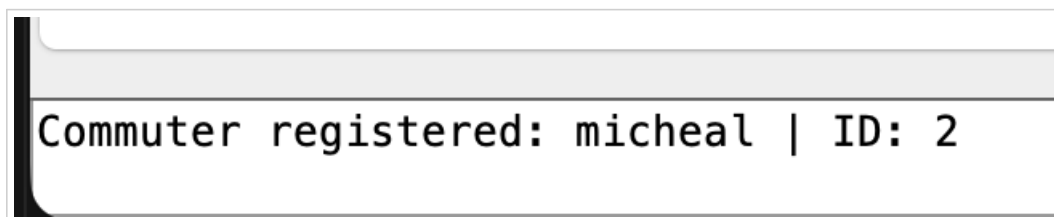
6.1 Registration and Ride Management

Figure 2: Commuter Registration



Successful registration of the first commuter.

Figure 3: Second Commuter Registration



Second commuter registered for testing ride requests.

Figure 4: Ride Offer

```
Ride offered: 101 | mumbai -> Thane
```

Ride 101 offered on the Mumbai to Thane route.

6.2 Request, Matching and Cost Splitting

Figure 5: Ride Request

```
Ride offered: 101 | mumbai -> Thane  
Ride request created: 1 | Mumbai -> Thane
```

Ride request created for the Mumbai to Thane route.

Figure 6: Route Matching

```
Ride request created: 1 | Mumbai -> Thane  
Ride matched for micheal | Ride ID: 101
```

The request is matched with Ride 101 using the compatible route.

Figure 7: Cost Splitting

```
Ride matched for micheal | Ride ID: 101  
Cost Split | Ride: 101 | Total: Rs. 600.0 | Per Person: Rs. 200.0
```

Ride 101: total ₹600, shared by three passengers at ₹200 per person.

6.3 Trip Tracking, Search and Reports

Figure 8: Trip Completion

```
Cost Split | Ride: 101  
Trip completed: 101
```

Ride 101 is marked as completed.

Figure 9: Trip History

```
TRIP HISTORY  
Ride 101 | samael | mumbai -> Thane
```

Completed trip history showing Ride 101.

Figure 10: Ride Search

```
Ride offered: 102 | mumbai -> Thane  
Ride found: 102 | samael | 11:00 | Rs. 400.0
```

Ride 102 is found by searching the Mumbai to Thane route.

6.4 Carpool Report Output

```
CARPPOOL REPORT
Registered Commuters: 2
Active Rides: 1
Ride Requests: 0
Completed Trips: 1

Rides sorted by cost:
Ride 102 - Rs. 400.0

Rides by departure time:
Ride 101 - 10:00
Ride 102 - 11:00
```

Figure 11: Carpool Reports

The report summarizes registered commuters, active rides, requests and completed trips, and displays rides ordered by cost and departure time.

7. Results and Conclusion

Results

- The seven expected modules are available from a single Swing interface.
- Commuter information, ride offers and ride requests can be created and managed.
- Compatible routes can be matched.
- Shared ride cost can be divided among passengers.
- Completed rides are maintained as trip history.
- Ride Search works using route information.
- Carpool Reports summarize the current records and ride ordering.
- The required Java concepts and collections are incorporated in the implementation.

Conclusion

The Ride Sharing Carpool Management System provides a focused implementation of Case Study 94. It combines Java classes, an interface, collections, CRUD operations, searching, sorting, validation, exception handling, multithreading and Swing to model the specified carpool workflow.

Project Scope

The application uses runtime Java collections for its records and is intended for demonstration and testing during program execution. No external database or unrelated service has been added.

8. References

1.	Case Study 94: Ride Sharing Carpool Management System, Java Programming, ITM Skills University.
2.	Supplied Java Case Study Report used as the report-format reference.
3.	Java source files and tested application output from the Carpool Management System project.

End of Report